UTM UNIVERSITI TEKNOLOGI MALAYSIA	**Software Engineering Department** **Faculty of Computing** **Universiti Teknologi Malaysia**

<<SDD Template>

Software Design Description (SDD)

<<Project Title>>

Prepared by: << Team Name >>

Document No.	**:**	**<<Document ID>>**
WPC Ref. No.	**:**	**<<Work Package Contract Ref. No.>>**
Revision	**:**	**<<Document Revision Number>>**
Date Issued	**:**	**<<Date>>**

Prepared by		**Reviewed by**		**Approved by**	
<< signature(s) >>		<< signature >>		<< signature >>	
Name(s)	<<Team Members>>	**Name**	<<Team Leader>>	**Name**	<< Lecturer >>
Position	Design Lead	**Position**	Design Manager	**Position**	Project Director

1. Frontispiece

a. Overview

Describe the content of the current version (referring to the current SD).

b. Issuing Organization / Team

State the organization / team's name.

c. Authorship / Project Team Members

List the team members in a table by stating their roles and the status for each assigned task (complete, partially complete, incomplete). If the assigned tasks are not done and have been assigned to other team members, state accordingly.

d. Revision History

REVISION NO.	ISSUE DATE	DETAILS OF REVISION
00	31/01/2023	Initial Submission
01	13/03/2023	Document updated from Preliminary Design of PAS…

Note: The title page should show the latest version, in the e.g. it should be 2.0. For your first submission (PR2-Chap.1 and 2), it should be 1.0, then 2.0 for PR3 and 3.0 for PR4. Primary author(s) could be the team leaders who should compile all materials for the purpose of SD submission. [remove this note in red texts and the examples in the table above]

Note:
This System Documentation (SD) template is adapted from IEEE Recommended Practice for Software Requirements Specification (SRS) (IEEE Std. 830-1998), Software Design Descriptions (SDD) (IEEE Std. 1016-1998 1), and Software Test Documentation (IEEE Std. 829-2008) that are simplified and customized to meet the need of SCSJ2203 course at Faculty of Computing, UTM. Examples of models are from Arlow and Neustadt (2002) and other sources stated accordingly.

Table of Contents

1	**Frontispiece**			1
2	**Introduction**			3
	2.1	Purpose		3
	2.2	Scope		3
	2.3	Context		3
	2.4	Summary		3
3	**References**			
4	**Glossary**			
5	**Design Body**			
	5.1	Design Stakeholders and Their Concerns		
	5.2	Context Viewpoint		
		5.2.1	Design Concerns	
		5.2.2	Design View (Use Case Diagram)	
	5.3	Structure Viewpoint (Architecture Style Diagram (Assignment 1))		
		5.3.1	Design Concerns	
		5.3.2	Design View (Class / Package / Component Diagram)	
	5.4	Information Viewpoint (DB Design)		
		5.4.1	Design Concerns	
		5.4.2	Design View (ERD Diagram)	
	5.5	Interface Viewpoint		
		5.5.1	Design Concerns	
		5.5.2	Design View (Interfaces UI/UX or Wireframe)	

	5.6	Logical Viewpoint (Creational & Structural & Behavioral Pattern (Assignment 2))		
		5.6.1	Design Concerns	
		5.6.2	Design View (Class Diagram)	
	5.7	Interaction Viewpoint (Sequence Diagram by Module / Sub-Module)		
		5.7.1	Design Concerns	
		5.7.2	Design View (Sequence Diagram)	
	5.8	Algorithm Viewpoint (For complex processing)		
		5.8.1	Design Concerns	
		5.8.2	Design View (Activity Diagram / Decision Tables / Flow Chart)	
6	**User Interface Design**			
7	**Appendices**			

2. Introduction

The following sections of the System Design Description (SDD) should provide the details of the entire system design. Remove the notes in read texts including these notes.

2.1 Purpose

This subsection should:

a) *Delineate the purpose of the SD;*
b) *Specify the intended audience for the SD.*

This SD describes…

2.2 Scope

This subsection should:

a) *Identify the software product(s) to be produced by name (e.g., Host DBMS, Report Generator, etc.);*
b) *Explain what the software product(s) will, and, if necessary, will not do;*
c) *Describe the application of the software being specified, including relevant benefits, objectives, and goals;*
d) *Be consistent with similar statements in higher-level specifications (e.g., the system requirements specification), if they exist.*

The software product is…

2.3 Context

This subsection should:

a) *Identify this to be used by whom*
b) *A guide for designer in the selection, organization, and presentation of design information*
c) *For whom to be practices*

This standard is intended for technical and managerial stakeholders who prepare and use SDDs. It guides designer in the selection, organization, and presentation of design information. For an organization developing its own SDD practices, the use of this standard can help to ensure that design descriptions are complete, concise, consistent, interchangeable, appropriate for recording design experiences and lessons learned, well organized, and easy to communicate…

2.4 Summary

This subsection should:

a) *Describe what the rest of the SD contains;*
b) *Explain how the SD is organized.*

Give an overview of the content of this SD document.

3. References

This section should:

a) *Provide a complete list of all documents referenced elsewhere in the SD;*
b) *Identify each document by title, report number (if applicable), date, and publishing organization;*
c) *Specify the sources from which the references can be obtained.*

Specify complete list of references using a standardized reference format.

4. Glossary

This section should provide the definitions of all terms, acronyms, and abbreviations used in the SD.

For the purposes of this standard, the following terms and definitions apply. The Authoritative Dictionary of IEEE Standards Terms [B13] and IEEE Std 12207™-2008 [B21] should be referenced for terms not defined in this clause

Definitions of all terms, acronyms and abbreviation used are to be defined here.

5. Design Body

This section of SDD should have the identified design stakeholders, design concerns and design viewpoints. The selected design viewpoint should come with the rationale behind it.

5.1 Design Stakeholders and Their Concerns

This should specify the following:

a) *The design stakeholders for the design subject.*
b) *The design concerns of each identified design stakeholder.*

An SDD shall address each identified design concern.

5.2 Context Viewpoint (Use Case Diagram)

5.2.1 Design Concerns

The Context viewpoint depicts services provided by a design subject with reference to an explicit context. That context is defined by reference to actors that include users and other stakeholders, which interact with the design subject in its environment. The Context viewpoint provides a "black box" perspective on the design subject.

This should specify the following:

a) *The purpose of the Context viewpoint is to identify.*
b) *The system features.*

The purpose of the Context viewpoint is to identify a design subject's offered services, its actors (users and other interacting stakeholders), to establish the system boundary and to effectively delineate the design subject's scope of use and operation. …

5.2.2 Design View

The system features include…

[Include use case diagram here – see example of Online Shopping below, source: https://www.uml-diagrams.org/index-examples.html]

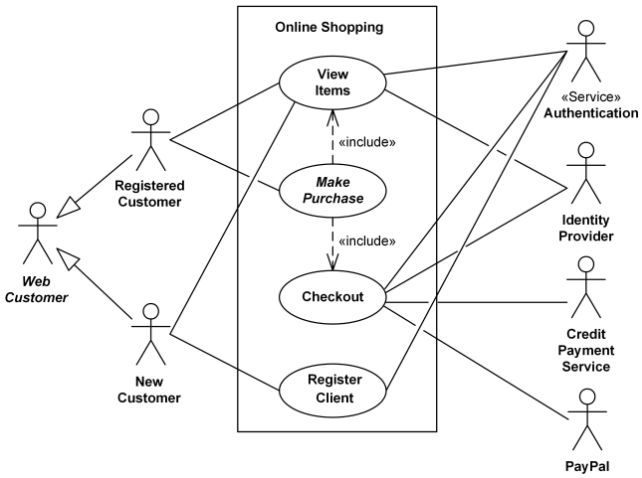


Figure 2.1: Use Case Diagram for <Name of the System>

5.2.2.1 UC001: Use Case <Name of Use Case 1>

[Provide code for each use case such as UC001 and so on… Include Use Case Description for each use case as in the example below. If there are any alternative flows, add the rows accordingly. Otherwise, just remove the rows. Ensure each use case has its own unique ID and the name of the heading above corresponds to the name of use case in the use case diagram (refer to Figure 2.1).

List out all scenario in the use case in Alternative flow section (these scenario will be used in Section 4 to draw the sequence diagram

Table 2.1: Use Case Description for <Name of Use Case>

Use case: <Name of Use Case>
ID: UCxxx
Actors:
Preconditions:
Flow of events: 1. 2. 3. …
Postconditions:
Alternative flow *n*:
Postconditions:
Exception flow (if any):

5.2.2.2 UC002: Use Case <Name of Use Case 2>

5.2.2.3 UC003: Use Case <Name of Use Case 3>

5.2.2.4 UC*n*: Use Case <Name of Use Case *n*>

…

5.3 Structure Viewpoint (Architecture Style (Assignment 1))

5.3.1 Design Concerns

The Structure viewpoint is used to document the internal constituents and organization of the design subject in terms of like elements (recursively).

Structure viewpoint informs user and the developer about the interaction between the classes / packages in the system…

5.3.2 Design View (Class / Package / Component or any UML Diagram)

Include the overall class diagram of your system here. While the Logical Viewpoint focuses on functional abstractions (classes and objects), the Structure Viewpoint provides a more detailed, architectural look at how these elements are physically or logically organized as distinct parts.

Figure 4.1: Class Diagram for <Name of the System>

5.4 Information Viewpoint (Database Design)

5.4.1 Design Concerns

The Information viewpoint is applicable when there is a substantial persistent data content expected with the design subject. Key concerns include persistent data structure, data content, data management strategies, data access schemes, and definition of metadata.

Explain how the information domain of your system is transformed into data structures. Describe how the major data or system entities are stored, processed and organized. List the database(s) or data storage items. For a small system, there is normally only one database. It consists of all the tables in which for object-oriented they are classes/objects. Use table for the listing.

The major data or systems entities are stored into a relational database named as… comprises *n* entities …

Entity Name	Description

5.4.2 Design View (ERD)

Alphabetically list the system entities or major data along with their types and descriptions (class/object, attributes). Focus on classes in domain layer; omit the controller/handler class. Use tables for easy listing.

Then show how the data structure relationship (Entity Relationship Diagram (ERD)). Describe the relation of one entity interact with the others. For a small system, there is normally only one database. It consists of all the tables in which for object-oriented they are classes/objects. Use table for the listing:

Data Dictionary

Entity: <First Entity>

Attribute Name	Type	Description

Entity: <Second Entity>

Attribute Name	Type	Description

Entity Relationship Diagram (ERD)

5.5 Interface Viewpoint (Interface Design)

5.5.1 Design Concerns

The Interface viewpoint provides information designers, programmers, and testers the means to know how to correctly use the services provided by a design subject. This description includes the details of external and internal interfaces not provided in the SRS. This viewpoint consists of a set of interface specifications for each entity.

Describe the functionality of the system from the user's perspective. Explain how the user will be able to use your system to complete all the expected features and the feedback information that will be displayed for the user.

The interface viewpoint cover user interfaces, hardware interfaces and software interfaces descriptions…

5.5.2 Design View (Wireframe design / UI design)

User Interfaces

This should specify the following:

a) *The **logical characteristics** of each interface between the software product and its users.*
*This includes those **configuration characteristics** (e.g., required screen formats, page or window layouts, content of any reports or menus, or availability of programmable function keys) necessary to accomplish the software requirements.*

b) *All the aspects of **optimizing the interface** with the person who must use the system.*
This may simply comprise a list of do's and don'ts on how the system will appear to the user. One example may be a requirement for the option of long or short error messages. Like all others, these requirements should be verifiable, e.g. "A clerk typist grade 4 can do function X in Z min after 1 h of training" rather than "a typist can do function X." (This may also be specified in the Software System Attributes under a section titled Ease of Use.)

Describe how the system will interact with its users.

Hardware Interfaces

*This should specify the logical characteristics of each **interface between the software product and the hardware** components of the system. This includes configuration characteristics (number of ports, instruction sets, etc.). It also covers such matters as what devices are to be supported, how they are to be supported, and protocols. For example, terminal support may specify full-screen support as opposed to line-by-line support.*

Specify the logical characteristic here…

Software Interfaces

*This should specify the use of **other required software products** (e.g., a data management system, an operating system, or a mathematical package), **and interfaces with other application systems** (e.g., the linkage between an accounts receivable system and a general ledger system). For each required software product, the following should be provided:*

- *Name;*
- *Mnemonic;*
- *Specification number;*
- *Version number;*
- *Source.*

For each interface, the following should be provided:

- *Discussion of the purpose of the interfacing software as related to this software product.*
- *Definition of the interface in terms of message content and format. It is not necessary to detail any*

Well-documented interface, but a reference to the document defining the interface is required.

Specify here…

Communication Interfaces

This should specify the various interfaces to communications such as local network protocols, etc.

Specify here…

5.6 Logical Viewpoint (Creational & Structural & Behavioral Pattern (Assignment 2))

5.6.1 Design Concerns

The purpose of the Logical viewpoint is to elaborate existing and designed types and their implementations as classes and interfaces with their structural static relationships. This viewpoint also uses examples of instances of types in outlining design ideas.

This should specify the following:

a) *The classes and interfaces.*
b) *The relationship.*

The purpose of the logical viewpoint is to elaborate the implementation of classes with their relationship. …

5.6.2 Design View (Package / Component / Class Diagram)

Include class diagram to represent all classes in the respective subsystem/package. Include the controller classes.

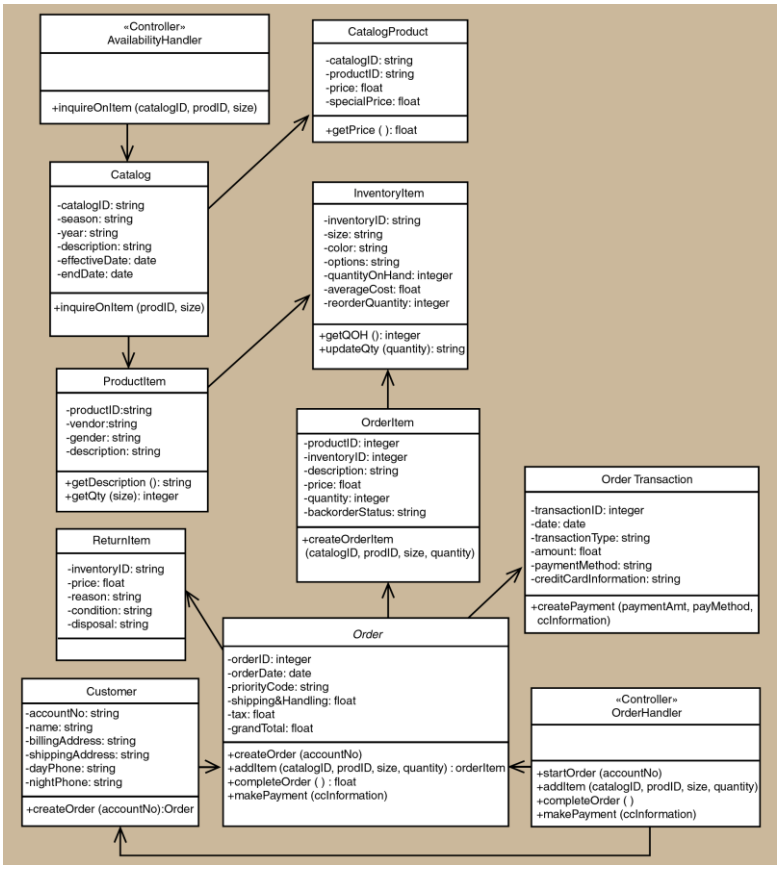


Figure 4.3: Class diagram for <Name of Package 1> Subsystem

5.7 Interaction Viewpoint (Sequence Diagram by Module / Sub-Module)

5.7.1 Design Concerns

The Interaction viewpoint defines strategies for interaction among entities, regarding why, where, how, and at what level actions occur.

… The functionalities of the system are given by the help of class and sequence diagrams. Moreover, it defines strategies for interaction among entities. …

5.7.2 Design View (Sequence diagram by Module / Sub-Module)

Include sequence diagram for each respective use case in your package. In these examples only sequence diagram Create New Phone Order Scenario and Cancel an Order Scenario are shown. Include the final sequence diagram that comprises view layer, controller and its problem domain (entity) and data access layer. Provide a unique code for each scenario of sequence diagram to be used in Section 7: Requirements Matrix.

a) SD001: Sequence diagram for Create New Phone Order

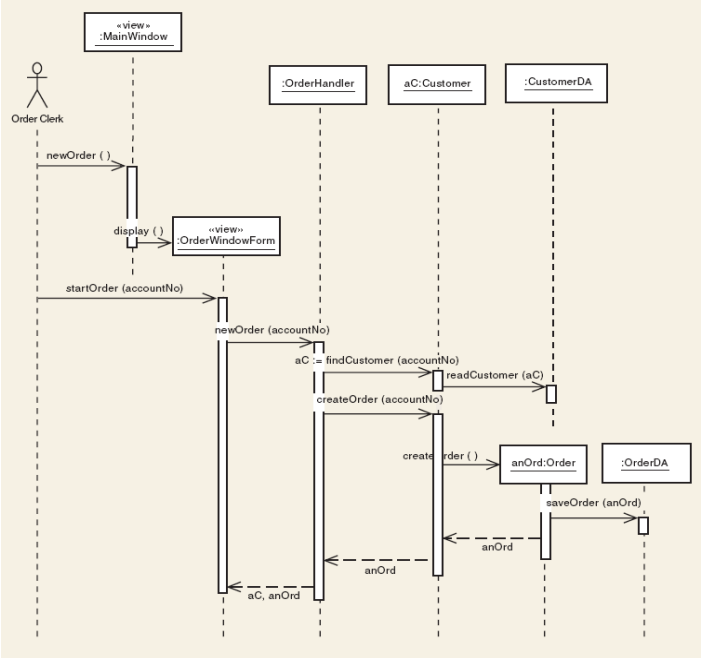


Figure 4.4: Sequence Diagram for <Create New Phone Order Scenario>

b) SD002: Sequence diagram for Create Cancel an Order Scenario

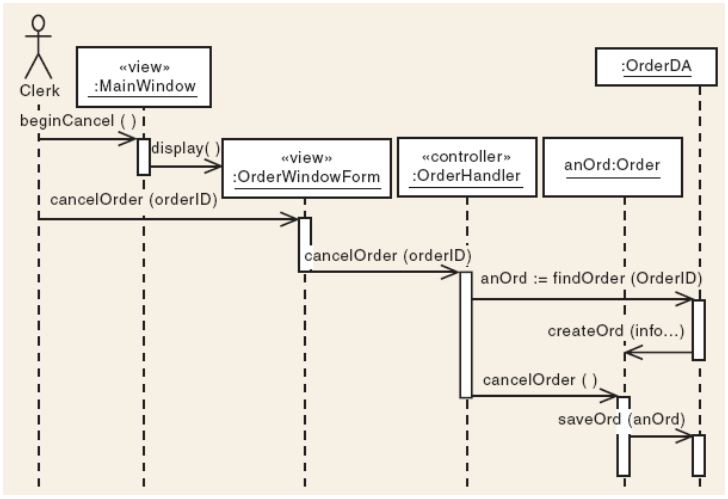


Figure 4.5: Sequence Diagram for <Cancel an Order scenario>

5.8 Algorithm Viewpoint (Construction Design For Complex Processing)

5.8.1 Design Concerns

The detailed design description of operations (such as methods and functions), the internal details and logic of each design entity.

The Algorithm viewpoint provides details needed by programmers, analysts of algorithms in regard to time-space performance and processing logic prior to implementation, and to aid in producing unit test plans…

5.8.2 Design View (Flowchart / Decision Table / State-Transition based / Activity Diagram)

Include the attribute descriptions for identification, processing and data for all design entities.

6. Appendices

Appendices may be included, either directly or by reference, to provide supporting details that could aid in the understanding of the Software Design Document.

Can provide detail design planning, gantt charts, or older version of UI/UX and wireframe design.